```c
// C program for implementation of Normal Bubble sort
#include <stdio.h>

void swap(int *xp, int *yp)
{
    int temp = *xp;
    *xp = *yp;
    *yp = temp;
}

// A function to implement bubble sort
void bubbleSort(int arr[], int n)
{
    int i, j;
    for (i = 0; i < n-1; i++) // loop for the number of trips
    {
        for (j = 0; j < n-i-1; j++) // loop for the number of comparisons per trip
        {
            if (arr[j] > arr[j+1]) //swap if the previous element is greater than the next one(for ascending order)
            {
                swap(&arr[j], &arr[j+1]);
            }
        }

    }
}

/* Function to print an array */
void printArray(int arr[], int size)
{
    int i;
    for (i=0; i < size; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

// Driver program to test above functions
int main()
{
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int n = sizeof(arr)/sizeof(arr[0]);
    printf("UnSorted array: \n");
    printArray(arr, n);
    bubbleSort(arr, n);
```

```
45:     printf("Sorted array: \n");
46:     printArray(arr, n);
47:     return 0;
48: }
49:
50: // Optimized implementation of Bubble sort
51: #include <stdio.h>
52:
53: void swap(int *xp, int *yp)
54: {
55:     int temp = *xp;
56:     *xp = *yp;
57:     *yp = temp;
58: }
59:
60: // An optimized version of Bubble Sort
61: void optimizedBubbleSort(int arr[], int n)
62: {
63:     int i, j,temp;
64:     int swapped;
65:     for (i = 0; i < n-1; i++) // loop for the number of trips
66:     {
67:         swapped = 0; //swapped is set to false for every trip
68:
69:         for (j = 0; j < n-i-1; j++) // loop for the number of
    comparisons per trip
70:         {
71:             if (arr[j] > arr[j+1]) // > for ascending order and <
    for descending order
72:             {
73:                 //swap(&arr[j], &arr[j+1]);
74:                 temp=arr[j];
75:                 arr[j]=arr[j+1];
76:                 arr[j+1]=temp;
77:                 swapped = 1;// if there is some swapping, swapped
    is set to true
78:             }
79:             printf("\ni=%d,j=%d,swapped=%d\n",i,j,swapped);
80:             printArray(arr, n);
81:         }
82:
83:         // IF no two elements were swapped by inner loop, then break
84:         if (swapped == 0)
85:             break;
86:     }
87: }
```

```
 88:
 89: /* Function to print an array */
 90: void printArray(int arr[], int size)
 91: {
 92:     int i;
 93:     for (i=0; i < size; i++)
 94:         printf("%d ", arr[i]);
 95:     printf("\n");
 96: }
 97:
 98: // Driver program to test above functions
 99: int main()
100: {
101:     int arr[] = {20,10,30,40,50};
102:     //int arr[] = {40,30,20,10};
103:     int n = sizeof(arr)/sizeof(arr[0]);
104:     printf("Unsorted array: \n");
105:     printArray(arr, n);
106:
107:     optimizedBubbleSort(arr, n);
108:     printf("Sorted array: \n");
109:     printArray(arr, n);
110:     return 0;
111: }
112:
113: // C program for implementation of selection sort
114: #include <stdio.h>
115:
116: void swap(int *xp, int *yp)
117: {
118:     int temp = *xp;
119:     *xp = *yp;
120:     *yp = temp;
121: }
122:
123: void selectionSort(int arr[], int n)
124: {
125:     int i, j, min_idx;
126:
127:     // One by one, move boundary of unsorted subarray
128:     for (i = 0; i < n-1; i++)
129:     {
130:         // Find the minimum element in unsorted array
131:         min_idx = i;
132:         for (j = i+1; j < n; j++)
133:         {
```

```
134:             if (arr[j] < arr[min_idx])
135:                 min_idx = j;
136:         }
137:
138:         // Swap the found minimum element with the first element
139:         swap(&arr[min_idx], &arr[i]);
140:     }
141: }
142:
143: /* Function to print an array */
144: void printArray(int arr[], int size)
145: {
146:     int i;
147:     for (i=0; i < size; i++)
148:         printf("%d ", arr[i]);
149:     printf("\n");
150: }
151:
152: // Driver program to test above functions
153: int main()
154: {
155:     int arr[] = {64, 25, 12, 22, 11};
156:     int n = sizeof(arr)/sizeof(arr[0]);
157:     selectionSort(arr, n);
158:     printf("Sorted array: \n");
159:     printArray(arr, n);
160:     return 0;
161: }
162:
163: //C program for insertion sort
164: #include <math.h>
165: #include <stdio.h>
166:
167: /* Function to sort an array
168: using insertion sort*/
169: void insertionSort(int arr[], int n)
170: {
171:     int i, key, j;
172:     for (i = 1; i < n; i++)
173:     {
174:         key = arr[i];
175:         j = i - 1;
176:
177:         /* Move elements of arr[0..i-1],
178:         that are greater than key,
179:         to one position ahead of
```

```
180:         their current position */
181:         while (j >= 0 && arr[j] > key)
182:         {
183:             arr[j + 1] = arr[j];
184:             j = j - 1;
185:         }
186:         arr[j + 1] = key;
187:     }
188: }
189:
190: // A utility function to print
191: // an array of size n
192: void printArray(int arr[], int n)
193: {
194:     int i;
195:     for (i = 0; i < n; i++)
196:         printf("%d ", arr[i]);
197:     printf("\n");
198: }
199:
200: // Driver code
201: int main()
202: {
203:     int arr[] = {12, 11, 13, 5, 6,11};
204:     int n = sizeof(arr) / sizeof(arr[0]);
205:
206:     insertionSort(arr, n);
207:     printArray(arr, n);
208:
209:     return 0;
210: }
211:
```